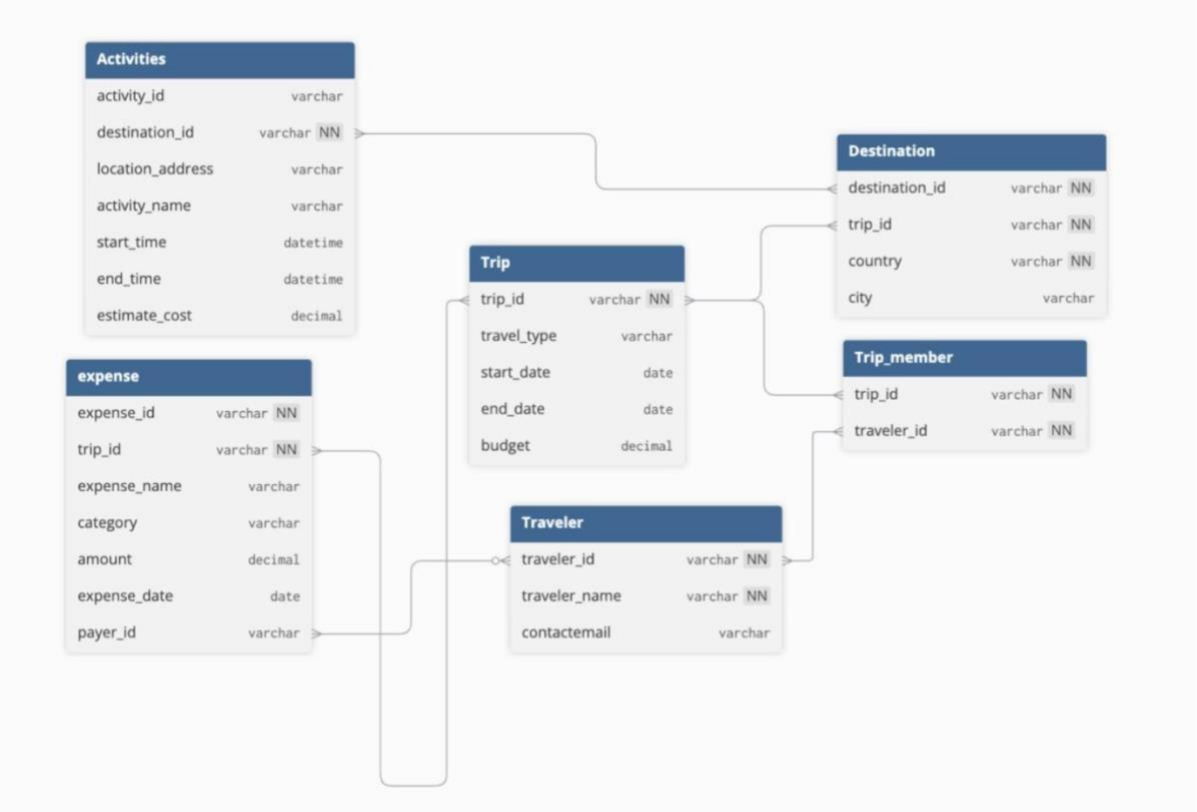
Schema Evolution and ER Modeling: Take the foundational two-table schema from your first check-in and expand it to fully support your mobile app's growing needs.



My Check-in 1 schema includes 6 tables.

To improve the functionality of organizing activities by day, I added a trip_date attribute to the Activities table. This allows users to plan daily itineraries and schedule specific activities for each day of their trips. I also added two category_name attribute to improve the functionality of searching activities by category. These attributes allow users to search for activities based on their preferred categories and add them to their daily itineraries more easily.

Activities table

Activity_id
Destination_id
Location_address
Activity_name
Start_time
End_time
Estimated_cost
Trip_date
Category_name

I also want to improve the feature of splitting bills, so I added 5 attributes to the Trip_member table.

Total_amount is the sum of all expense amounts having the same trip_id in the Expense table.

Split_count is the total number of members in each trip (the number of distinct traveler_id values having the same trip_id).

Split_amount is the amount each member needs to pay (Total_amount / Split_count).

Total_paid is the total amount each traveler has already paid (the sum of expense amounts having the same payer_id and trip_id).

Balance is the amount each traveler needs to pay or receive (Total_paid – Split_amount).

Trip_member table

Trip_id
Traveler_id
Total_amount
Split_count
Split_amount
Total_paid
Balance

Step-by-Step Normalization Demonstration: Walk me through how you normalize your database up to 3rd Normal Form (3NF). Please include the intermediate steps (1NF and 2NF) so I can see your progression. To make sure everything is working as intended, keep your sample data values and facts consistent throughout the steps—this helps demonstrate that breaking down the tables doesn't lose any information or change your schema's core purpose!

Trip_member table

Trip_id
Traveler_id
Total_amount
Split_count
Split_amount
Total_paid
Balance

Trip_id	Traveler_id	Total_amount	Split_count	Split_amount	Total_paid	Balance
TP01	TL01	300	3	100	150	+50
TP01	TL02	300	3	100	100	0
TP01	TL03	300	3	100	50	-50
TP02	TL01	800	2	400	500	+100
TP02	TL04	800	2	400	300	-100

The trip_member table is already in 1NF because each cell contains only 1 atomic value

2NF: Total_amount, split_count, split_amount depend only on Trip_id, while total_paid and balance depend on composite key (Trip_id, Traveler_id).

Trip_finance table

Trip_id	Total_amount	Split_count	Split_amount
TP01	300	3	100
TP02	800	2	400

Trip_member table

Trip_id	Traveler_id	Total_paid	Balance
TP01	TL01	150	+50
TP01	TL02	100	0
TP01	TL03	50	-50
TP02	TL01	500	+100
TP02	TL04	300	-100

3NF = 2NF because there are no transitive dependencies

The Problem-Solving Journey: Map out the functional dependencies in your data, and share some examples of insertion, deletion, or update anomalies you encountered in your early designs. Then, explain how your final 3NF structure resolves them. Please include a brief reflection on your troubleshooting process. I value seeing the mental effort and how you worked through the challenges to untangle the design just as much as the final diagram itself.

Functional dependency:

(Trip_id) -> (Total_amount, Split_count, Split_amount)

(Trip_id, Traveler_id) -> (Total_paid, Balance)

Some examples of insertion, deletion, or update anomalies

Example1: Add other travelers to one trip (adding TL04 to TP01) => when I update Trip_member, I have the problem of data inconsistency because

Trip_id	Traveler_id	Total_amount	Split_count	Split_amount	Total_paid	Balance
TP01	TL01	300	3 -> 4	100 -> 75	150	update
TP01	TL02	300	3 -> 4	100 -> 75	100	update
TP01	TL03	300	3 -> 4	100 -> 75	50	update
TP01	TL04	300	4	75	update	update
TP02	TL01	800	2	400	500	+100
TP02	TL04	800	2	400	300	-100

Joining the same trip (TP01), 4 travelers have different split_amount and split_count. To resolve this problem I need to update these two attributes and recalculate Balance attribute of all three existing travelers in TP01. Therefore, six cell values need to be updated.

However, after nomalization, if I want to add TL04 to TP01 I just need to change the split_count and split_amount only one time in Trip_finance table and insert TL04 in Trip_member table. This also helps recalculate Balance attribute in Trip_member table more easily.

Trip_finance table

Trip_id	Total_amount	Split_count	Split_amount
TP01	300	**4**	**75**
TP02	800	2	400

Trip_member table

Trip_id	Traveler_id	Total_paid	Balance
TP01	TL01	150	update
TP01	TL02	100	update
TP01	TL03	50	update
TP01	TL04	Update	update
TP02	TL01	500	+100
TP02	TL04	300	-100

Example2: The total_amount of TP01 increases from 300 to 1200.

Trip_id	Traveler_id	Total_amount	Split_count	Split_amount	Total_paid	Balance
TP01	TL01	300 -> 1200	3	100 -> 400	update	update
TP01	TL02	300 -> 1200	3	100 -> 400	update	update
TP01	TL03	300 -> 1200	3	100 -> 400	update	update
TP02	TL01	800	2	400	500	+100
TP02	TL04	800	2	400	300	-100

There are also 6 cell values needed to be updated. However, after normalization, I just need to change the total_amount once in Trip_finance table and then update Trip_member table.

Trip_id	Total_amount	Split_count	Split_amount
TP01	1200	3	400
TP02	800	2	400

A brief reflection on your troubleshooting process

I figured out that Total_amount, Split_count, and Split_amount are repeated many times in the original Trip_Member table. This is a data redundancy problem. Therefore, when I need to update, insert, or delete information related to a trip or its travelers, I need to update multiple values to maintain data consistency. After identifying the functional dependencies, I decomposed the original Trip_Member table into Trip_Member and Trip_Finance. As a result, I not only reduced data redundancy and inconsistency but also made it easier to identify the travelers belonging to each trip, calculate the number of travelers in each trip, and manage financial information for both the entire trip and individual travelers.